Performance-Based Dynamic Load Balancer

for a Secure Persistent Group Chat

Student: Paritosh Lahre

Roll number: 12341550

Course / Lab: CSD, Lab 6 — Dynamic Load Balancing and Persistent Chat

Submitted endpoint: `http://10.1.75.53:5277`

Repository:
`https://github.com/SandstromPL/CSD-Dynamic-Load-Balancing-and-Persistent-Chat`

Report date: 15 September 2026

Part I is a compact, self-contained report of what was built, how it is architected, and what it measured – everything needed for evaluation. Part II, placed after the conclusion, is the full chronological experiment log: every candidate tried, every flag tuned, and the measured reason each decision was made.

Contents

Abstract **3**

AI Usage Disclosure **3**

I Compact Evaluation Report **4**

1 Problem statement and objectives **4**

2 Assigned systems and ports **4**

3 Architecture and algorithm used for the leaderboard session **5**
3.1 Dynamic scoring algorithm 5

4 Official leaderboard result **6**

5 How this result was reached, in brief **8**

6 Distributed WebSocket broadcast **8**

7 Final configuration reference **9**

8 Code structure **10**

9 Local experimental results **10**

10 Limitations **11**

11 Conclusion **11**

II Detailed Experimental Log **13**

12 Code blocks covering assignment requirements **13**
12.1 Dynamic, performance-based backend selection (`load-balancer/main.go`) 13
12.2 `POST /message`: idempotent insert and duplicate/conflict detection (`backend/server.py`) 14
12.3 `GET /feed` and automatic health detection (`backend/server.py`) 15
12.4 Retained, now cluster-distributed, secure chat (`backend/message_bus.py`) . . . 15

13 Complete configuration reference **16**

14 Progressive implementation and experimental history **18**
14.1 Starting point: one secure persistent chat server 18
14.2 First extension: HTTP API, shared persistence, three backends 18
14.3 Dynamic load balancer and the first hosted failures 18
14.4 Moving PostgreSQL off the load-balancer host 18
14.5 Making the local generator resemble the evaluator 19
14.6 Bounded-resource and feed-delivery experiments 19
14.7 How the recorded leaderboard result was achieved 19
14.8 Post-leaderboard completion: distributed live WebSocket delivery 20

15 Optimisation experiments and decisions **20**

16 Test 23 in detail: static latency win, deterministic listener failure 21
16.1 The client-cap fix, validated but not re-submitted 22

17 Failure analysis and limitations, in full 22
17.1 The dominant unresolved bottleneck: complete-feed delivery under concurrency . 22
17.2 Complete limitations list 22

18 Verification 23

Short final summary 24

A Figure reproduction 24

B Code and experiment traceability 24

C Historical: the removed commit-serialisation diagnostic 24

Abstract

This project extends a secure, persistent group-chat application from one server to three backend servers behind a performance-based dynamic load balancer (DLB). All backends share one PostgreSQL database, so a message written through any backend is durable and visible through every backend. A unique message ID with a database uniqueness constraint makes retries idempotent, and message text is encrypted at rest with AES-256-GCM. After the scored leaderboard session, the previously process-local secure WebSocket chat was extended with a distributed event bus so that a message committed through one backend is broadcast live to clients connected to a *different* backend.

Six official leaderboard submissions are reported. Test 18 is the best complete, balanced result (1.07% static error, 758.07 ms mean; 38,445/ 40,000 breakpoint successes). Test 23, the final submission, reached the lowest static latency of the project (566.83 ms) but suffered a complete listener-level collapse at the 2,500-user breakpoint stage, caused by a connection-cap configuration defect that was diagnosed and fixed afterward but not re-submitted. The recorded assessment was 12 marks at mean rank 13.

AI Usage Disclosure

AI coding assistants were used throughout this project for planning, implementing and reviewing code, and writing this report; all engineering decisions and reported results were verified by me.

Part I

Compact Evaluation Report

1 Problem statement and objectives

The previous application was a secure WebSocket group chat with persistent, encrypted, signed messages running as a single process. Lab 6 required the backend to be deployed across three allotted systems and exposed through one load-balancer URL, while retaining the existing chat semantics, and to add:

- `POST /message` and `GET /feed`;
- dynamic, performance-based backend selection rather than fixed round robin;
- automatic health detection and failover;
- a shared persistent database with unique message IDs and duplicate-write prevention; and
- a custom load generator producing response-time and Sys1–Sys4 utilisation plots.

The engineering priority order was **correctness first, availability second, performance third**: no change that weakened idempotency, feed integrity, or chat semantics was accepted merely because it measured faster in one run.

2 Assigned systems and ports

System	Role	Private port	Public mapped port
Sys1	HTTP DLB	5000	5277
Sys1	WSS DLB	7000	7277
Sys2	HTTP backend	6000	6278
Sys2	WSS backend	5000	5278
Sys3	HTTP backend	6000	6279
Sys3	WSS backend	5000	5279
Sys3	PostgreSQL	4000	4279
Sys4	HTTP backend	6000	6280
Sys4	WSS backend	5000	5280

Table 1: Deployment port map. Only the Sys1 HTTP and WSS public URLs are client entry points; direct backend ports were used solely for diagnosis.

Each allotted system runs under a 512 MiB memory cgroup. Sys1 hosts only the load balancer; Sys3 co-hosts one backend process *and* PostgreSQL, which makes it structurally the second most memory-constrained system after Sys1 (Section 9).

3 Architecture and algorithm used for the leaderboard session

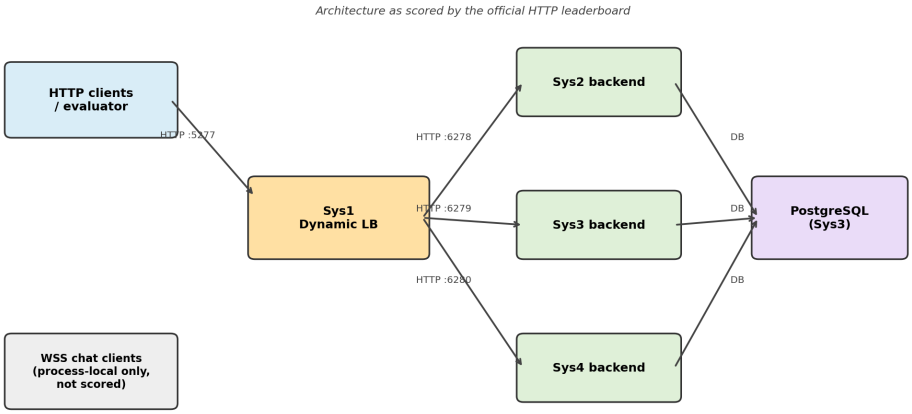


Figure 1: Architecture exactly as scored by the official HTTP leaderboard. WSS chat existed but was process-local only during this session (no cross-backend broadcast); Section 6 describes the distributed extension added afterward.

Sys1 accepts every public HTTP connection, selects a backend, reserves one capacity slot, and reverse-proxies the request. The backend batches message inserts, encrypts message text, and commits to PostgreSQL before returning success. `GET /feed` incrementally refreshes an in-process serialised snapshot from committed rows and streams the complete JSON body.

3.1 Dynamic scoring algorithm

Each backend is independently health-checked and publishes statistics every 500 ms (`/internal/stats`: CPU, active-request count, average latency, database-queue and memory figures). From that, the DLB computes one performance score per backend:

$$\text{score} = 0.25 \times \text{CPU}\% \;+\; 0.45 \times \text{active-request saturation}\% \;+\; 0.30 \times \text{latency saturation}\% \quad (1)$$

where active-request saturation is $\min(100, \text{active}/100 \times 100)$ and latency saturation is $\min(100, \text{latency}/100\text{ms} \times 100)$ – i.e. 100 concurrent requests or 100 ms average latency each count as "fully loaded" on that dimension. **Why these weights:** this workload is dominated by PostgreSQL round trips and large feed transfers rather than CPU-bound computation, so CPU alone is a weak real-time saturation signal – a backend can be CPU-idle while its queue of outstanding requests grows because it is waiting on the database or a slow client. Active-request count is therefore weighted highest (0.45): it reflects what a backend is *currently* committed to, not just its recent history. Latency (0.30) adds a second, independent signal that active-request count alone can miss – for instance, a backend can have few outstanding requests but still be slow per request because of database contention. CPU (0.25) is kept as a smaller, corroborating term rather than removed, since a sustained CPU spike (e.g. from concurrent AES-GCM encryption or JSON serialisation) is still a genuine cost the balancer should react to.

For the latency term, the DLB uses whichever is *larger*: the backend's own self-reported average latency, or the DLB's directly *observed* proxy latency from its own request timing. This closes a gap where a backend's self-report could lag reality (for example, because its own metrics loop is itself delayed under load) – the balancer never trusts a backend to under-report its own slowness. The observed component decays with a five-second exponential half-life once it stops being refreshed, so that one historically slow request does not permanently penalise a backend that

has since recovered; without this decay, a single 10-second outlier could keep a healthy backend artificially excluded long after the outlier passed.

Selection procedure:

1. Exclude any backend that is unhealthy, exceeds the working-memory or hard total-memory exclusion limits, or already holds 200 outstanding capacity reservations. These are hard safety constraints, checked before any score is even consulted.
2. Among the remaining *safe* backends, form the *eligible* set whose score (Equation 1) is below the configured threshold (60).
3. Pick the eligible backend with the fewest currently outstanding requests – a shortest-queue policy, which minimises tail latency better than a purely score-based pick when backends have similar scores but different real-time queue depths. Rotate round-robin among exact ties so that identical scores do not always favour the same backend.
4. If *every* safe backend exceeds the threshold (all are "busy"), fall back to the least-outstanding *safe* backend rather than rejecting the request outright – fallback only relaxes the *soft* performance threshold from step 2, never the hard safety exclusions from step 1.
5. Reserve the chosen backend's capacity slot with an atomic compare-and-swap; if a concurrent goroutine already won that exact slot (the score/outstanding-count snapshot used for selection can be momentarily stale under concurrency), retry selection from step 1 rather than double-booking capacity.

This is what makes the balancer *dynamic* rather than fixed round robin: both the eligible set and the chosen backend change continuously as health and measured performance change, and round robin is used only as the last-resort tiebreaker after every performance, memory and queue-depth comparison has already been evaluated and found equal. Health probes (`/health`, polled every second with a 2.5 s timeout, two consecutive failures to mark down) use a database connection dedicated away from the write pool – sharing that pool was the project's first major hosted failure: under write load, health probes queued behind message writes, timed out, and caused perfectly healthy backends to be marked down, which looks identical to a real outage from the balancer's point of view unless the two are given independent resources (Part II, Section 14).

Persistence, ordering and security are summarised here and detailed further in Part II: PostgreSQL's `UNIQUE(message_id)` constraint with `ON CONFLICT DO NOTHING` makes retries idempotent; message text is AES-256-GCM encrypted with the message ID as associated data; and the load balancer performs WSS TLS pass-through, so it never decrypts chat traffic.

4 Official leaderboard result

Test	Static success	Static error	Static mean	Breakpoint success	Breakpoint error	Breakpoint mean	Complete?
16	19,774/20,000	1.13%	779.92 ms	38,378/40,000	4.06%	1,126.44 ms	Yes
18	19,786/20,000	**1.07%**	758.07 ms	38,445/40,000	3.89%	1,137.77 ms	Yes
20	19,735/20,000	1.33%	754.70 ms	38,386/40,000	4.04%	1,192.69 ms	Yes
21	19,761/20,000	1.20%	586.91 ms	32,342/35,000	6.49%	1,208.97 ms	No
22	17,099/17,261	0.94%	1,346.21 ms	34,735/36,912	5.90%	1,775.96 ms	No
23	19,684/20,000	1.58%	**566.83 ms**	33,557/40,000	16.11%	875.77 ms	Yes*

Table 2: Official leaderboard results. *Test 23 issued its full request budget on both boards, but the breakpoint board's final stage collapsed completely (Part II, Section 16); its 16.11% aggregate error is not comparable to Tests 16–20.

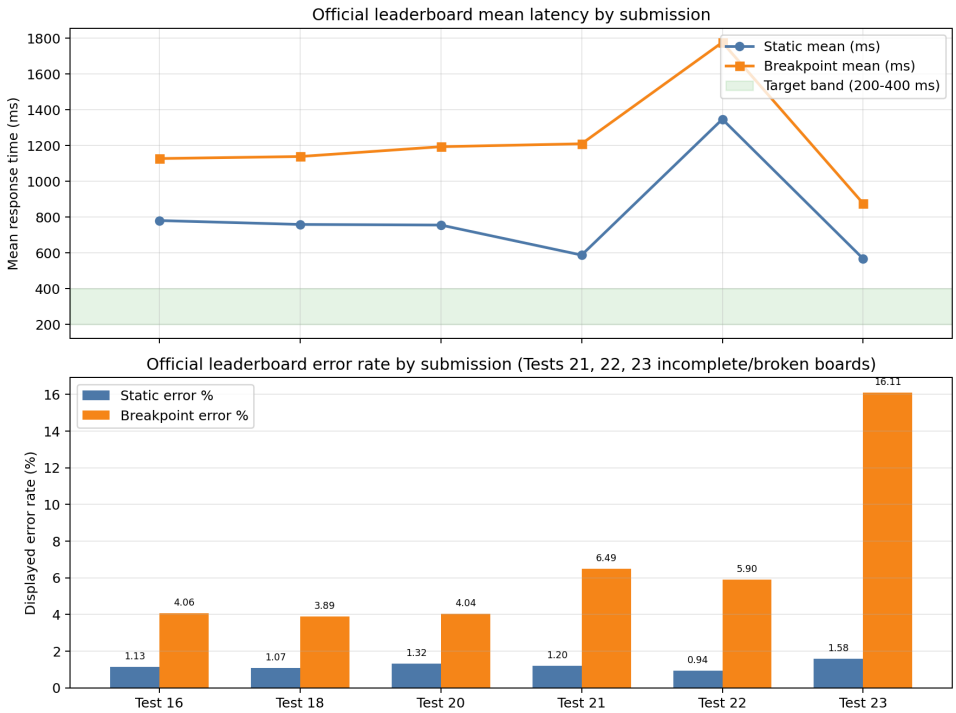


Figure 2: Official mean latency and error rate across all six submissions. Static latency trends down overall; breakpoint error and latency both spike on the two submissions (21, 23) that used an under-qualified candidate. Neither board entered the 200–400 ms target band.

Test 18 is the best balanced, fully complete result. Test 23 — the final submission, using the client-capacity fix described in Section 3 plus a retuned 82% memory guard — achieved the project's best static latency (566.83 ms) with all requests issued on both boards, but its breakpoint board failed completely at the 2,500-user stage (Figure 3): 0 of 5,000 requests succeeded, due to a connection-cap configuration defect rather than a memory, database, or backend-capacity failure. This is analysed in full, with the validated fix, in Part II, Section 16.

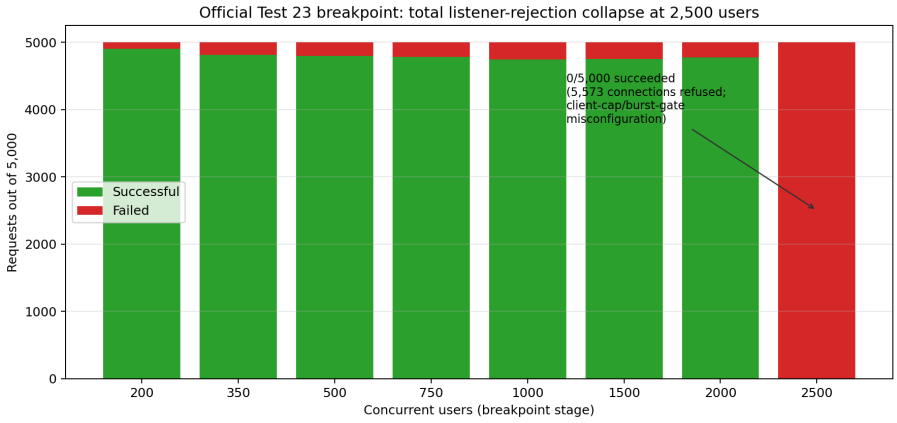


Figure 3: Official Test 23 breakpoint board: every stage from 200 through 2,000 users completed normally; the 2,500-user stage failed completely.

5 How this result was reached, in brief

The final configuration was not the starting point. In outline: an initial naive deployment suffered false health failures and repeated out-of-memory kills; dedicated health probing and least-outstanding fallback fixed availability; PostgreSQL was relocated off the load-balancer host; the local load generator was rebuilt to reproduce the evaluator's real connection volume, after a configuration passed local testing and still broke an official submission; message and feed traffic were separated into independently capped resource classes once feed transfers were found to be the dominant bottleneck; five successive load-balancer feed-cache designs were tried and rejected; and the final client-capacity and memory-guard values were reached through matched, repeated qualification runs rather than single favourable results. **Part II (after the conclusion) gives the complete chronological account, including every rejected candidate, the measured reason it was rejected, and how each final flag value in Table 3 was actually reached.**

6 Distributed WebSocket broadcast

After the scored HTTP session concluded, chat semantics were tested across users connected to *different* backend processes, which the HTTP-only leaderboard never exercises. Shared PostgreSQL made message history common to all three backends, but did not itself push a *live* message to a socket connected to a different backend. This gap was closed by adding a batched PostgreSQL `LISTEN/NOTIFY` event bus.

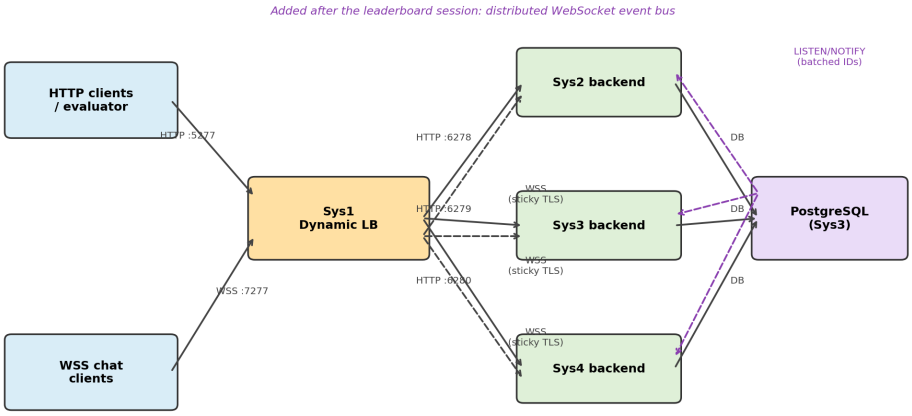


Figure 4: Architecture after the distributed WebSocket extension. After a commit, its ID is published in a batch of at most 32 IDs; each backend with locally connected clients performs one ordered query per batch and broadcasts locally.

The first design published one `NOTIFY` per message through a fixed 4,096-entry queue; under combined load it dropped events and reached 83.28 s worst-case cross-cluster delivery, with Sys3 memory coming within 64 KiB of its hard limit. Batching fixed this decisively:

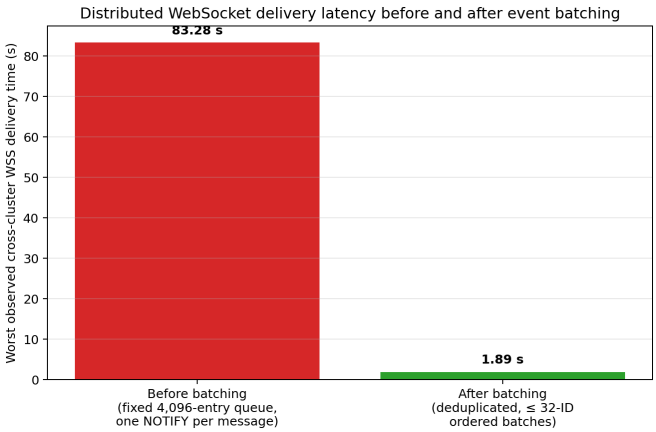


Figure 5: Worst observed cross-cluster delivery latency before and after batching `NOTIFY` events into ordered groups of $\leq$32 IDs.

A bounded, repeatable combined test (12 WebSocket clients through the DLB plus one direct observer per backend, concurrent with a 250-user/2,000 -message HTTP burst) delivered 120/120 expected probe copies at 253.09 ms mean / 1.89 s worst-case latency, with zero dropped events and zero memory-guard activity. This validates correctness and latency at 12–15 concurrent clients; production-scale slow-client isolation remains future work (Section 10).

7 Final configuration reference

Flag / variable	Final value	Purpose
`-threshold`	60	Maximum performance score for normal eligibility.
`-max-memory` / `-max-total-memory`	90% / 98%	Backend working-set / hard total-memory exclusion.
`-lb-max-memory`	82%	Sys1 memory percentage that activates shedding.
`-backend-cap`	200	Concurrent message-proxy connections per backend.
`-proxy-max-feed-conns-per-host`	4	Concurrent feed-proxy connections per backend.
`-client-max-connections`	2,600	Flat hard client-connection cap.
`-client-connection-burst`	0	Guarded burst reserve (disabled after Test 23; Section 16.1).
`-socket-read/write-buffer-bytes`	16,384	General/message socket buffer size.
`-proxy-feed-read` / `-client-feed-write-buffer-bytes`	131,072	Feed-specific socket buffers.
`-proxy-buffer-bytes`	32,768	Reverse-proxy copy buffer size.
`GOMEMLIMIT` / `GOGC`	192 MiB / 100	Go runtime memory limit / GC target on the load balancer.
`DB_POOL_MIN` / `DB_POOL_MAX`	1 / 6	Per-backend PostgreSQL pool bounds.
`DB_WRITE_BATCH_SIZE` / `_WORKERS`	64 / 1	Batched write worker sizing.
`FEED_CACHE_CHUNK_BYTES`	65,536	Backend-local feed serialisation chunk size.

Table 3: Selected final configuration (curated subset). How each value was reached is detailed in Part II, Section 15; every flag and environment variable that exists, not just this subset, is listed in Part II, Section 13.

8 Code structure

Used by the scored leaderboard system	
`backend/server.py`	HTTP protocol, AES-GCM encryption, PostgreSQL/SQLite persistence, batched writes, incremental feed serialisation, metrics.
`load-balancer/main.go`	HTTP dynamic selection, reverse proxying, health/statistics monitoring, admission and memory safety.
`load-generator/main.go`	Deterministic staged workloads, response validation, telemetry, CSV/JSON export.
`scripts/integration-test.sh`	Three-backend correctness, duplicate-message, failover and feed integration checks.
Added for the distributed broadcast system (post-leaderboard)	
`backend/message_bus.py`	Batched PostgreSQL `LISTEN/NOTIFY` transport for committed live messages.
`load-balancer/wss_proxy.go`	Sticky WebSocket backend selection and TLS pass-through.
`scripts/wss-cluster-load-test.py`	Bounded combined HTTP+WebSocket qualification harness.

Table 4: Repository component structure, split by what the leaderboard score actually exercised.

9 Local experimental results

The representative local run (`client2600-abrupt2500-repeat`, the `leaderboard-paired-v2` profile, seed `20260911`, `fresh-user` connections, 5% feed mix) issued 60,000 requests across the static (250–1,000 user) and breakpoint (200–2,500 user) ladders. All 56,970 message writes succeeded with zero duplicates, mismatches or ordering errors; every strict failure was a large `/feed` transfer reaching the ten-second deadline.

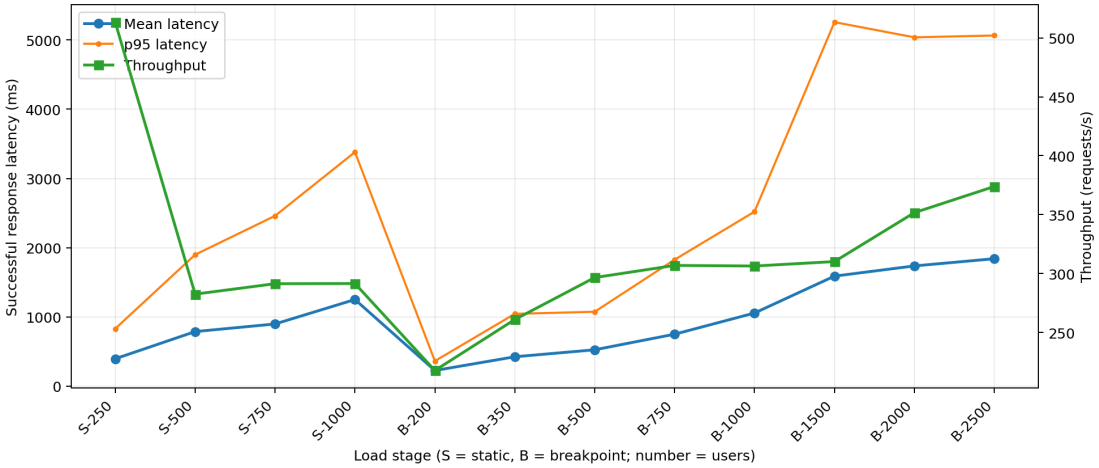


Figure 6: Mean/p95 successful-response latency and throughput by stage.

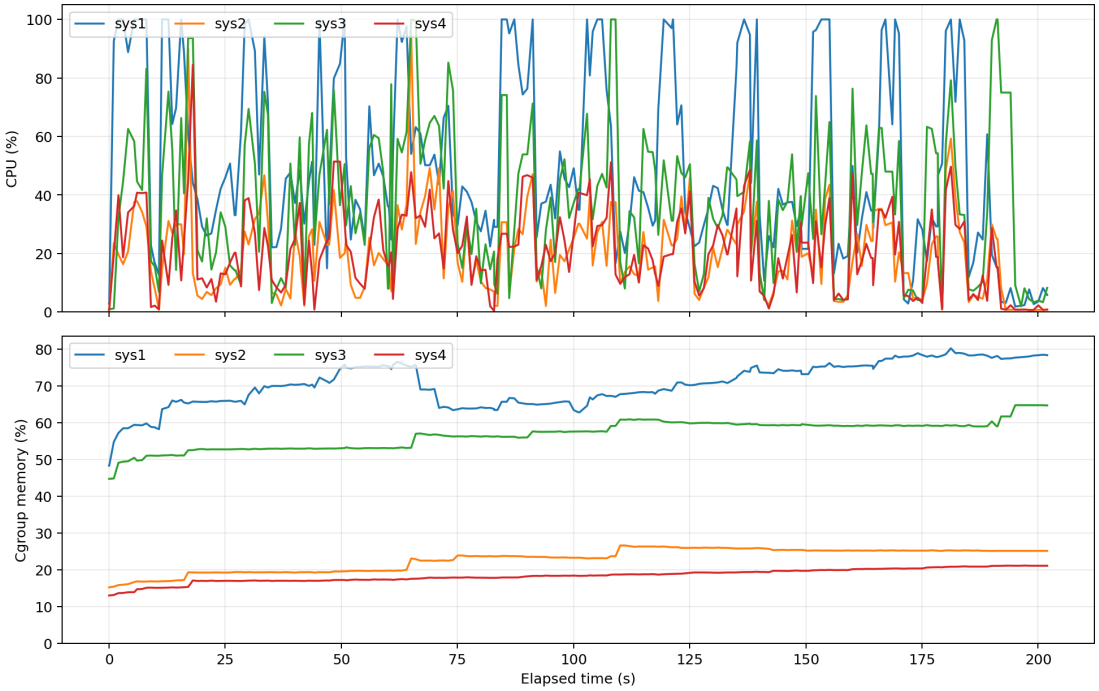


Figure 7: CPU and cgroup memory for all four systems. Sys1 carries proxy and socket cost; Sys3 carries a backend process and PostgreSQL together. All four systems stayed below their 512 MiB cgroup ceiling throughout.

10 Limitations

The dominant unresolved bottleneck is delivery of the complete, ever-growing `/feed` history under concurrency (reaching ∼27.5 MB by the end of a full run) – message writes remained reliable at every tested concurrency, while concurrent feed requests degraded. `/feed` returns the entire history on every call, with no pagination contract; WebSocket fan-out was validated only at 12–15 concurrent clients, not at the 1,200+ scale a full deployment might need; and hosted measurements carry real run-to-run noise (over 100 ms observed between identical runs). Part II, Section 17 gives the complete, measurement-backed analysis of why the feed bottleneck resists every mitigation tried.

11 Conclusion

The project fulfils the core Lab 6 architecture: one public dynamic load balancer, three independently monitored backends, shared encrypted persistent chat data, idempotent message insertion, automatic failover, the required HTTP routes, a custom measurement tool, and a fully retained — and, after the leaderboard, cluster-distributed — secure group chat.

The largest, most durable gains came from fixing system-level failure modes – dedicated health probes, least-outstanding fallback, atomic capacity reservations, feed/message traffic separation, and connection-model fidelity between local testing and the real evaluator – not from payload trimming or increasingly elaborate scheduling, several variants of which were tried and rejected (Part II). The experiments repeatedly confirm one clear structural boundary: returning the entire feed under rising concurrency is an $O(\text{history size} \times \text{concurrent readers})$ operation against fixed serving capacity, and no connection cap, socket buffer, or caching policy tested removes that asymptotic cost. The distributed WebSocket work completes the system: PostgreSQL provides durable, retrievable history, while the batched event bus makes a commit live on every

connected backend, without ever replaying old history to a new joiner.

Part II

Detailed Experimental Log

This part is the complete, chronological account behind every claim, number and configuration value in Part I: every candidate that was designed, measured, and either kept or rejected, in the order it actually happened, with the specific evidence behind each decision. It opens with the actual code that satisfies the assignment's requirements and the complete configuration reference, before returning to the chronological narrative.

12 Code blocks covering assignment requirements

Every snippet below is quoted verbatim from the current repository (not paraphrased), and each is mapped to the specific Lab 6 requirement it satisfies. Together with Part I, Section 3, they show the actual mechanism behind every architectural claim in this report.

12.1 Dynamic, performance-based backend selection (`load-balancer/main.go`)

The scoring formula (Equation 1) as implemented:

```
score := 0.25*cpu + 0.45*inflightLoad + 0.30*latencyLoad
if stale {
    score = min(100, score+15) // penalize backends with old statistics
}
```

The selection procedure described in Section 3, including the hard safety exclusions, the soft-threshold eligible set, the least-outstanding tiebreak, the safety-preserving fallback, and the atomic-reservation retry loop that handles a lost race against another concurrent request:

```
func (lb *LoadBalancer) selectBackend() (*Backend, ScoreDetails, bool) {
    for range len(lb.backends) + 1 {
        safe := make([]candidate, 0, len(lb.backends))
        eligible := make([]candidate, 0, len(lb.backends))
        for _, backend := range lb.backends {
            if !backend.isAlive() {
                continue
            }
            item := candidate{backend: backend, details: backend.score(lb.config,
                time.Now()),
                outstanding: backend.inflight.Load()}
            // Memory and the explicit backend cap are safety constraints, not
            // score preferences. Never bypass them in fallback mode.
            if item.details.MemoryOverLimit ||
                (lb.config.BackendCap > 0 && item.outstanding >= lb.config.BackendCap)
                    {
                continue
            }
            safe = append(safe, item)
            if item.details.BelowLoadThreshold {
                eligible = append(eligible, item)
            }
        }
        if len(safe) == 0 {
            return nil, ScoreDetails{}, false // no healthy, memory-safe backend at
                all
```

```
        }
        fallback := len(eligible) == 0
        pool := eligible
        if fallback {
            pool = safe // every safe backend is over threshold: fall back, do not
                reject
        }
        // Prefer the backend with the least immediate outstanding work; rotate ties.
        best := leastOutstanding(pool)
        chosen := best[int(lb.tie.Add(1)-1)%len(best)]
        if chosen.backend.tryReserve(lb.config.BackendCap) {
            if fallback {
                lb.metrics.Fallbacks.Add(1)
            }
            return chosen.backend, chosen.details, fallback
        }
        // Lost the reservation race to a concurrent request: retry from scratch.
    }
    return nil, ScoreDetails{}, false
}
```

12.2 `POST /message`: idempotent insert and duplicate/conflict detection (`backend/server.py`)

```
async def message_handler(request: web.Request) -> web.Response:
    data = await parse_message_input(request)
    client_name, message, message_id = validate_message(data, request)
    ciphertext, nonce = encrypt_message(message_id, message)
    stored, inserted = await STORAGE.insert_message(
        message_id, client_name, ciphertext, nonce, None, None, "http", utc_now(),
    )
    if inserted and MESSAGE_BUS is not None:
        MESSAGE_BUS.publish(stored.message_id) # announce to the cluster event bus
    if inserted:
        existing_message = {"message-id": stored.message_id, "client-name": stored.
            client_name,
            "msg": message, "timestamp": stored.created_at,
            "sequence": stored.sequence, "source": stored.source}
    else:
        existing_message = stored.public() # decrypt the already-committed row
    if not inserted and (existing_message["client-name"] != client_name
                         or existing_message["msg"] != message):
        return json_response({"status": "conflict", "error":
            "message-id already belongs to a different message",
            "message-id": message_id}, 409)
    return json_response({"status": "created" if inserted else "duplicate",
        "duplicate": not inserted, "message": existing_message}, 201 if inserted else
             200)
```

The uniqueness itself is enforced one layer down, by the database – not by this handler – so it holds even under concurrent retries routed to *different* backends:

```
WITH input AS (
    SELECT * FROM UNNEST($1::text[], $2::text[], $3::bytea[], ...)
    WITH ORDINALITY AS t(message_id, client_name, ciphertext, ..., position)
),
deduplicated AS (SELECT DISTINCT ON (message_id) * FROM input ORDER BY message_id,
    position)
```

```
INSERT INTO messages (message_id, client_name, ciphertext, ...)
SELECT message_id, client_name, ciphertext, ... FROM deduplicated
ORDER BY position
ON CONFLICT (message_id) DO NOTHING
RETURNING sequence, message_id, created_at;
```

12.3 GET /feed and automatic health detection (backend/server.py)

```
async def feed_handler(request: web.Request) -> web.StreamResponse:
    snapshot = await FEED_CACHE.snapshot() # incremental, per-backend serialised
        cache
    etag = f'"feed-{snapshot.sequence}-{snapshot.count}"'
    response = web.StreamResponse(status=200, headers={
        "Content-Type": "application/json; charset=utf-8",
        "X-Backend": BACKEND_ID, "ETag": etag,
        "X-Feed-Sequence": str(snapshot.sequence), "X-Feed-Count": str(snapshot.count
            )})
    response.content_length = snapshot.content_length
    if not await prepare_feed_response(response, request):
        return response
    await write_feed_body(response, snapshot)
    return response


async def health_handler(_: web.Request) -> web.Response:
    try:
        healthy = await asyncio.wait_for(STORAGE.ping(), timeout=HEALTH_CHECK_TIMEOUT
            )
    except Exception:
        LOG.exception("database health check failed")
        healthy = False
    return json_response({"status": "healthy" if healthy else "unhealthy",
        "backend": BACKEND_ID}, 200 if healthy else 503)
```

The load balancer polls exactly this endpoint every second and removes a backend from routing after two consecutive non-200 responses, implementing the required automatic failover.

12.4 Retained, now cluster-distributed, secure chat (backend/message_bus.py)

The batching fix from Section 14.8: outgoing message IDs are coalesced and published in groups of at most 32, and the whole publish loop reconnects and backs off on any database error rather than crashing the backend process:

```
async def _publisher(self) -> None:
    while True:
        try:
            await self.outgoing_ready.wait()
            self.outgoing_ready.clear()
            if not self.outgoing:
                continue
            pending = list(self.outgoing)[: self.batch_size] # batch_size <= 32
            if self.publisher_connection is None or self.publisher_connection.
                is_closed():
                self.publisher_connection = await self._connect()
            await self.publisher_connection.execute(
                "SELECT pg_notify($1, $2)", self.channel,
                json.dumps(pending, separators=(",", ":")))
```

```
            for message_id in pending:
                self.outgoing.pop(message_id, None)
            if self.outgoing:
                self.outgoing_ready.set() # more queued than fit in one batch: go
                    again
        except asyncio.CancelledError:
            raise
        except Exception:
            self.logger.exception("could not publish WebSocket message events")
            if self.publisher_connection is not None:
                with contextlib.suppress(Exception):
                    self.publisher_connection.terminate()
                self.publisher_connection = None
            self.outgoing_ready.set()
            await asyncio.sleep(0.5)
```

Each backend's matching listener consumes batches the same way and performs one ordered database fetch per batch before broadcasting locally, which is what turned the 83.28 s worst-case delivery of the original one-`NOTIFY` -per-message design into the 1.89 s worst case reported in Section 6.

13 Complete configuration reference

Table 3 in Part I gives the curated, final-value subset that matters most for reproducing the scored result. This section lists *every* flag and environment variable the deployed code actually reads, taken directly from source rather than from documentation, so it cannot go stale the way a hand-maintained list can.

Table 5: Every `load-balancer/main.go` flag (35 total), with its in-code default. Final deployed values that differ from the default are noted in the purpose column; see Section 15 for why.

Flag	Default	Purpose
`-listen`	`:8080`	Public load-balancer listen address.
`-backends`	(required)	Comma-separated backend base URLs.
`-threshold`	75	Maximum score for normal eligibility. *Deployed: 60.*
`-max-memory`	95%	Backend working-set memory exclusion limit. *Deployed: 90%.*
`-max-total-memory`	98%	Backend hard total-cgroup memory exclusion limit.
`-lb-max-memory`	80%	Sys1 memory percentage activating shedding. *Deployed: 82%.*
`-lb-max-socket-bytes`	160 MiB	Sys1 TCP socket bytes activating early shedding; 0 disables.
`-max-inflight`	100	Active requests treated as 100% request load.
`-admission-limit`	0	Hard concurrent `POST /message` limit; 0 disables.
`-backend-cap`	0	Hard concurrent request cap per backend; 0 disables. *Deployed: 200.*
`-backend-queue-timeout`	10 s	Maximum wait for a memory-safe backend slot.
`-latency-target`	100 ms	Latency treated as 100% latency load.
`-stats-interval`	500 ms	Backend statistics poll interval.
`-health-interval`	1 s	Backend health poll interval.
`-monitor-timeout`	2.5 s	Health/statistics request timeout.
`-backend-timeout`	15 s	Backend response-header timeout.
`-failure-limit`	2	Consecutive failures before marking a backend down.
`-proxy-max-idle`	256	Maximum total idle proxy connections retained.
`-proxy-max-idle-per-host`	64	Maximum idle proxy connections retained per backend.
`-proxy-max-conns-per-host`	200	Maximum total proxy connections to each backend.
`-proxy-max-feed-conns-per-host`	4	Maximum feed-only proxy connections to each backend.
`-proxy-idle-timeout`	10 s	Idle timeout for backend proxy connections.

Flag	Default	Purpose
`-client-idle-timeout`	5 s	Idle timeout for incoming client connections.
`-client-max-connections`	2,200	Maximum simultaneously accepted client connections; 0 disables. *Deployed: 2,600.*
`-client-connection-burst`	400	Additional guarded slots below memory/socket margins. *Deployed: 0 (Section 16.1).*
`-client-new-timeout`	2 s	Minimum age before an idle `StateNew` socket is reclaimed at the cap.
`-socket-read-buffer-bytes`	16 KiB	Requested TCP receive buffer, client/backend sockets.
`-socket-write-buffer-bytes`	16 KiB	Requested TCP send buffer, client/backend sockets.
`-proxy-feed-read-buffer-bytes`	0	Feed backend receive buffer; 0 inherits the general value. *Deployed: 128 KiB.*
`-client-feed-write-buffer-bytes`	0	Client send buffer while serving a feed; 0 inherits the general value. *Deployed: 128 KiB.*
`-proxy-buffer-bytes`	32 KiB	Pooled reverse-proxy copy buffer size.
`-proxy-feed-buffer-bytes`	0	Feed-only copy buffer size; 0 inherits `-proxy-buffer-bytes`.
`-wss-listen`	(disabled)	Public TCP address for TLS WebSocket pass-through. *Added post-leaderboard.*
`-wss-backends`	(disabled)	Backend `host:port` list, same order as `-backends`. *Added post-leaderboard.*
`-wss-max-connections`	512	Maximum concurrent proxied WSS connections. *Added post-leaderboard.*

Table 6: Every environment variable `backend/server.py` reads (32 total), with its in-code default.

Variable	Default	Purpose
`BACKEND_ID`	hostname	Backend identity used in logs and the `X-Backend` header.
`HTTP_HOST` / `HTTP_PORT`	`0.0.0.0` / 6000	HTTP bind address.
`WS_HOST` / `WS_PORT`	`0.0.0.0` / 5000	WebSocket bind address.
`ENABLE_WEBSOCKET`	1	Toggles the WSS listener on this backend.
`ENABLE_CLUSTER_WEBSOCKET`	0	Toggles the distributed event bus (Section 6).
`WS_EVENT_CHANNEL`	`chat_message_events`	PostgreSQL `LISTEN/NOTIFY` channel name.
`WS_EVENT_QUEUE_MAX`	4,096	Upper bound on coalesced pending event IDs (batches are $\leq$32).
`TLS_CERT` / `TLS_KEY`	sample paths	WSS certificate/key files.
`DATABASE_URL`	(empty)	PostgreSQL DSN; empty selects SQLite.
`SQLITE_PATH`	`data/chat.db`	SQLite file path when `DATABASE_URL` is empty.
`HEALTH_CHECK_TIMEOUT`	1.5 s	Timeout wrapping the database ping in `/health`.
`HISTORY_LIMIT`	50	Messages sent to a newly joined WebSocket client.
`MAX_CLIENT_NAME`	100	Maximum `client-name` length.
`MAX_MESSAGE_BYTES`	64 KiB	Maximum message body size.
`HTTP_SOCKET_READ/WRITE_BUFFER_BYTES`	32 KiB	Backend HTTP listener socket buffers.
`LOG_LEVEL`	`INFO`	Python logging level.
`MESSAGE_ENCRYPTION_KEY`	(empty)	Base64 AES-256 key; preferred over the key file.
`MESSAGE_ENCRYPTION_KEY_FILE`	`data/encryption.key`	On-disk AES key fallback, shared across all backends.
`DB_HEALTH_TIMEOUT`	1 s	Dedicated health-connection timeout.
`DB_WRITE_BATCH_SIZE`	64	Rows per batched insert transaction.
`DB_WRITE_BATCH_DELAY_MS`	2 ms	Batching window while the write queue is non-empty.
`DB_WRITE_BATCH_WORKERS`	1	Batched write workers per backend.
`DB_WRITE_QUEUE_MAX`	256	Write queue capacity before backpressure.
`DB_WRITE_QUEUE_TIMEOUT`	5 s	Maximum wait for a queue slot before `503`.
`DB_POOL_MIN` / `DB_POOL_MAX`	1 / 6	asyncpg connection pool bounds.
`DB_COMMAND_TIMEOUT`	10 s	asyncpg per-command timeout.

Variable	Default	Purpose
`FEED_CACHE_CHUNK_BYTES`	64 KiB	Backend-local incremental feed serialisation chunk size.

14 Progressive implementation and experimental history

14.1 Starting point: one secure persistent chat server

The previous lab's application combined a TLS WebSocket listener, RSA-PSS signed client messages, AES-GCM encrypted local persistence, and process-local broadcast to connected sockets. It was a working single-server group chat, but it exposed none of the required HTTP routes and had no notion of distributing load across hosts.

14.2 First extension: HTTP API, shared persistence, three backends

`POST /message` and `GET /feed` were added to the existing `aiohttp` process alongside the WSS listener. All three deployed backend processes ran identical application code and pointed at one shared PostgreSQL database; a single externally-supplied AES key was distributed to all three so that any backend could decrypt rows written by any other. The database's `UNIQUE(message_id)` constraint, combined with `INSERT ... ON CONFLICT (message_id) DO NOTHING`, made retried writes safe without any backend-side coordination.

This phase established *shared persistent visibility*: a message inserted through Sys2 could immediately be read back from `/feed` through Sys3 or Sys4. It deliberately did *not* yet make live, process-local WebSocket clients connected to those other backends receive the message immediately – that distinction is addressed in Section 14.8.

14.3 Dynamic load balancer and the first hosted failures

A Go reverse proxy on Sys1 became the sole public HTTP entry point, scoring backends on CPU, active-request count and observed latency. Early maximum-load testing exposed two systemic problems: (1) health checks shared the busy PostgreSQL write pool, so probes queued, timed out, and marked perfectly healthy backends down, generating thousands of spurious `503`s; (2) client sockets, proxy connections and Go runtime memory were unbounded, repeatedly exceeding the 512 MiB cgroup limit and triggering kernel OOM kills.

In one early 20,000-request test, the original policy completed only 17,013 requests at 14.94% error, almost entirely false `503`s. A *dedicated* health database connection and a 2.5-second monitor timeout raised the identical workload to 19,961 successes at 0.19% error; least-outstanding-request fallback then completed 20,000/20,000. These were the first decisive correctness/availability fixes and were never revisited.

14.4 Moving PostgreSQL off the load-balancer host

Sys1 was simultaneously accepting every client connection, proxying potentially multi-megabyte feed responses, *and* hosting PostgreSQL in the earliest deployment layout. Its cgroup recorded repeated OOM kills under load. PostgreSQL was relocated to Sys3 (which also runs one backend process), while all three backends continued to share the same database instance and encryption key. This reduced Sys1's competing working set and materially improved DLB availability under load.

14.5 Making the local generator resemble the evaluator

The first custom load-generator profiles reused HTTP connections far more aggressively than the official evaluator does, which made some *local* successes misleading: at a nominal "2,000 user" stage, one official run held roughly 1,600–1,900 simultaneous connections, where the equivalent local run only produced 240–300. A configuration that passed two clean, matched local repeats under the old connection model subsequently broke an official submission's breakpoint board outright, purely because local testing never reproduced the real connection volume.

The Go generator was rebuilt with staging ladders matching the evaluator, per-stage-distinct message IDs, a `fresh-user` mode that retains one persistent HTTP session per virtual user, a "held new connections" mode reproducing the `net/http StateNew` pressure failure, separate header-vs-body accounting, and per-second resource sampling. This closed the largest known fidelity gap; local qualification became necessary, though still not sufficient, before any official submission.

14.6 Bounded-resource and feed-delivery experiments

Under sustained mixed load, a pattern was confirmed repeatedly: **short message writes remained reliable at every tested concurrency, while complete `GET /feed` transfers degraded sharply as history grew** (a populated feed reached ∼27.5 MB). Message and feed traffic were therefore split into independently capped resource classes: feed proxy connections capped at 4 per backend versus 200 message reservations; TCP buffers split 16 KiB general / 128 KiB feed-specific (found by sweeping 16–256 KiB); copy buffers fixed at 32 KiB; feed capacity reservations released *after backend selection* rather than after response headers, once telemetry showed ∼7.8 s feed header waits occupying a shared reservation the whole time (Section 16); cgroup guards separating reclaimable file cache from true working-set memory; and explicit Go runtime bounds (`GOMEMLIMIT=192MiB`, `GOGC=100`).

Many further candidates were designed, measured, and reverted: compression, larger buffers, six backend processes, feed-aware/topology-aware routing, a reduced feed schema, and five successive feed-cache designs. Each improved *some* local metric in isolation but caused guard activity, incomplete breakpoint stages, or non-reproducible results (full list in Section 15). The complete feed schema and a direct-proxy baseline were ultimately restored for final deployment.

14.7 How the recorded leaderboard result was achieved

Leaderboard Tests 16–20 established the most stable band of the project. Test 18 was the best balanced, complete result. Test 21 substantially reduced static mean latency using an early feed-cache candidate, but that candidate's memory guard rejected 655 requests at the 2,000-user breakpoint stage, and the evaluator stopped the board early once aggregate error exceeded 20%. Test 22 disabled the load-balancer feed cache entirely; while its *displayed* static error looked attractive, both boards under-issued their full request budgets against a per-stage safety deadline.

Following Test 22, the feed-cache implementation (1,776 tracked lines) was removed entirely, returning to the proven Test-18-era direct-proxy design. Three "exact recovery" repeats against this baseline did *not* reliably reproduce Test 18's numbers – static completion missed the 19,550 promotion gate in all three, and one suffered a 1,134-message transport failure at high concurrency. This led to the Section 16 diagnosis: feed requests silently consumed shared 200-slot message capacity while waiting on their own four-connection feed transport. Fixing that, and correcting the client cap from a 2,200+400-burst scheme to a flat 2,600, produced the final Test 23 submission.

14.8 Post-leaderboard completion: distributed live WebSocket delivery

After the scored session concluded, chat semantics were tested across users connected to *different* backends – a gap the HTTP-only leaderboard never exercises, corrected as separate, later work. The first distributed design published one `NOTIFY` per message through a fixed 4,096-entry queue; under a deliberately extreme combined test (∼47,000 HTTP messages each also broadcast to every connected WS client, ∼1.5M socket deliveries), the queues overflowed and dropped events, with worst-case delivery reaching 83.28 s and Sys3 memory within 64 KiB of its hard limit – a near-OOM failure, not a candidate for another repeat. The fix (batched, deduplicated `NOTIFY` groups of ≤32 IDs, one query per batch instead of per message) is detailed with code in Section 12 and its measured result in Section 6.

15 Optimisation experiments and decisions

Two methodological lessons, learned the hard way, shaped every experiment from roughly the midpoint of the project onward:

1. **Identical code produces materially different results on different runs.** Two runs of the byte-identical binary and configuration, reset to the same clean database state, were observed to differ by as much as ∼130 ms of mean latency and 30–50 completions out of 20,000/40,000. Every non-trivial candidate in this project's later stages was required to pass at least one matched repeat before promotion.
2. **The official evaluator does not resemble a naive local test harness** (Section 14.5). A local qualification pass is necessary but not sufficient evidence that an official submission will succeed.

Table 7: Every optimisation experiment, in approximate chronological order. Full numeric detail for each is preserved in `docs/optimization-log.md`.

Experiment	Outcome and decision
Dedicated health database path + 2.5 s monitor timeout	Removed false health failures; success rose from 17,013/20,000 to 19,961/20,000 on an early hosted test. **Kept.**
Least-outstanding fallback routing	Completed 20,000/20,000 on the early maximum-load test with peak reservations balanced 334/334/333 across backends. **Kept.**
Score threshold 65 vs. 75 (early write-only profile)	75 prevented excessive fallback and completed the early write-heavy profile; later retuned to 60 once the mixed message/feed profile was introduced. **Adapted.**
Message batch 2×32 vs. 1×64	A single 64-row batch worker repeated with zero message errors and higher throughput than two 32-row workers. **Kept (1×64).**
Feed proxy concurrency cap	High feed concurrency increased memory pressure and contended with message writes; four connections per backend protected the write path. **Kept (cap 4).**
Socket / copy buffer size sweeps	Larger general-purpose buffers increased cgroup memory pressure; 128 KiB was beneficial only on the feed-specific read-/write sides. **Kept split sizes.**
Reduced feed schema (drop sequence/timestamp/source)	Did not measurably improve throughput or latency and weakened returned chat semantics. **Rejected.**
Six HTTP backend processes	Added process and memory overhead with no repeatable gain over three. **Rejected; restored three.**
Multiple load-balancer feed-cache designs and schedulers	Several reduced latency in isolated local runs, but guard activity or breakpoint regressions appeared in matched repeats or official tests. **Rejected; feed cache fully removed (1,776 lines).**

Experiment	Outcome and decision
Concurrent-commit database write path	Wrapping the batched insert in an explicit transaction was briefly removed for a ∼20% isolated throughput gain, but reproducibly caused message timeouts at 2,000+ concurrent users under the realistic mixed workload. **Rejected; explicit transaction restored.**
PostgreSQL advisory-lock commit serialisation	Eliminated the measured out-of-order-commit feed-cache-reset pathology (11 resets / 266,326 retransferred rows → zero, reproducibly), but its write-serialisation cost erased the benefit above 2,000 concurrent users. **Kept as a diagnostic for one cycle, then removed entirely** (Appendix C).
Direct proxy without a load-balancer feed cache	Simpler, and retained for final stability; it did not eliminate the complete-feed bottleneck but removed dormant experimental complexity. **Kept as final baseline.**
Exclude queued feeds from general backend capacity	Released a feed's shared 200-slot reservation immediately after backend selection instead of after response headers. Reproducibly protected message integrity and improved breakpoint completion/latency across repeats; one repeat still exposed a guard-triggered collapse at 2,500 users. **Kept, with the downside explicitly documented.**
Sys1 memory-guard threshold 80% → 82%	Removed a known false-margin failure mode; validated across three clean legs with zero guard events even when the sampled peak crossed the old 80% line. **Kept.**
Client connection cap: 2,200+400 burst → flat 2,600	The burst-disable gate at 77% memory collapsed the effective cap to 2,200 during an abrupt concurrency jump, causing Test 23's breakpoint collapse. A flat 2,600 cap with no burst gate reproduced zero rejections across two repeats. **Kept; validated locally, not yet re-submitted.**
Batched cluster WebSocket events	Replaced fixed 4,096-entry per-message queues (74–83 s worst-case delivery) with ordered, deduplicated batches of ≤32 IDs (1.89 s worst-case in a bounded repeat). **Kept.**

16 Test 23 in detail: static latency win, deterministic listener failure

Test 23 used the "exclude queued feeds from general backend capacity" fix together with the retuned 82% Sys1 memory guard and the *original* (pre-fix) 2,200-base-plus-400-burst client connection scheme.

Static board: 19,684/20,000 successes (1.58% error), 566.83 ms mean, with 18,993/18,993 message writes delivered and 100% feed completeness/correctness – the best official static latency of the entire project, though its error rate is above the tight 1.07–1.33% band held by Tests 18–20.

Breakpoint board: stages at 200 through 2,000 users cleared normally, with completions of 4,901, 4,813, 4,796, 4,782, 4,741, 4,751 and 4,773 out of 5,000 respectively. The final, 2,500-user stage then returned **0 out of 5,000** successful requests, dragging the aggregate breakpoint result to 33,557/40,000 (16.11% error) at a misleadingly low 875.77 ms mean (instantaneous connection rejections are fast and pull the mean down, unlike a slow timeout).

Post-run telemetry identifies the exact mechanism, and it is a *configuration* defect, not a capacity or memory-safety failure: the Sys1 memory guard *never fired* during this stage. Instead, 5,573 client connections were rejected at the listener. The connection-cap scheme in effect provided a 2,200-connection base cap plus a 400-connection "guarded burst" reserve, but that burst reserve is automatically disabled once Sys1 memory reaches `lb-max-memory` − 5 percentage points – 77% under the 82% guard, a conservative margin intended to leave headroom before the real safety boundary. The abrupt jump to 2,500 concurrent clients did not leave enough

reclaimable idle connections for the listener to free up room within that 77% budget, so the effective connection ceiling collapsed from 2,600 to 2,200 at precisely the moment 2,500 clients arrived – a self-inflicted shortfall, not an exhaustion of real memory or backend capacity. Feed verification after the run confirmed correctness was entirely unaffected: 52,326 unique, correctly ordered messages, zero duplicates, zero ordering errors.

16.1 The client-cap fix, validated but not re-submitted

The diagnosed fix is `client-max-connections=2600` with `client-connection-burst=0` – keeping the validated 2,600 hard ceiling, but removing all dependency on the premature 77% burst-disable gate, so the 82% memory guard remains the *only* safety boundary governing connection admission. Two local qualification legs, each deliberately reproducing the abrupt-2,500-user transition with 500 freshly opened, request-less sockets injected immediately before that stage, confirmed the fix:

- **Leg 1:** peak occupancy 2,501 clients; the focused 2,500-user stage completed 4,730/5,000 (5.4% error) – directly reversing Test 23's 0/5,000 collapse. All 56,970 message responses succeeded; zero listener rejections; Sys1 peaked at 80.49%.
- **Leg 2 (repeat):** peak occupancy 2,581 clients; final stage completed 4,721/5,000; all message operations succeeded; 1,707 idle connections were reclaimed cleanly with zero stale-new closures and zero rejections; Sys1 peaked at 80.25%.

This causally validates the diagnosis: the old scheme's effective 2,200 cap rejected 5,573 connections and zeroed the final stage; the corrected flat 2,600 cap admits the same peak client population with zero rejection and normal final-stage completion, in two independent repeats. This fix was not re-submitted to the official evaluator before the project deadline, and is reported here as validated-but-unsubmitted rather than as an achieved leaderboard result.

17 Failure analysis and limitations, in full

17.1 The dominant unresolved bottleneck: complete-feed delivery under concurrency

Every failure mode investigated – feed-cache designs, socket buffer sweeps, connection-cap tuning, backend-capacity accounting, the PostgreSQL commit-ordering fix – traces back to one constraint: `GET /feed` returns the *entire* history as one JSON body, and that body grows without bound (∼27.5 MB by the end of a representative run). Message writes stayed reliable at every tested concurrency; concurrent *feed* requests either waited for headers or were interrupted mid-body.

Independent measurements converge on the same conclusion. Direct backend-to-backend feed transfers were consistently faster than the identical workload through Sys1, pointing at proxy/-socket contention rather than encryption or serialisation cost – an idle feed capture completed in under one second even against a fully populated history. Phase-level instrumentation showed ∼88% of the delay on a cached feed path was *queueing* for a delivery slot, not transmission once one was acquired – the constraint is concurrent service *capacity*, not speed.

17.2 Complete limitations list

- `/feed` returns the entire history on every request; pagination or an incremental/delta contract would reduce the asymptotic transfer cost as history grows, but the assignment's required route contract did not provide for such a change without breaking evaluator compatibility.

- Sys3 hosts both PostgreSQL and one backend process, so it has structurally less memory headroom than Sys2 or Sys4.
- The live WebSocket notification path is recoverable through explicit history request, but is not itself a durable, per-client acknowledgement stream.
- WebSocket broadcast cost scales as $U \times R$ socket deliveries for U connected users and R new messages per second; 1,200 or more simultaneous WebSocket clients were never validated.
- Slow-client isolation and bounded, per-client delivery queues on the WebSocket fan-out path remain future work; the current batched design was validated at 12–15 concurrent clients, not at full evaluator scale.
- The PostgreSQL advisory-lock fix provably eliminated the feed-cache out-of-order-commit reset pathology, but its own write-path serialisation cost outweighed its benefit above roughly 2,000 concurrent users, so it was removed from the deployed tree entirely rather than shipped disabled – Appendix C preserves the mechanism for the record.
- Hosted measurements contain real environmental noise – identical code and configuration, reset to the same clean state, were observed to vary by over 100 ms of mean latency between runs.

18 Verification

The final code passed the following checks before being considered for deployment or official submission:

- Go load-balancer and load-generator unit test suites, including race-detector-enabled runs (`go test -race ./...`) and `go vet`.
- 26 Python backend and message-bus unit tests (`python3 -m unittest`).
- The complete three-backend shell integration script (`scripts/integration-test.sh`), covering three initially healthy backends, HTTP message creation and shared feed visibility through a different backend, fifty concurrent identical-ID retries producing exactly one stored row, TLS WebSocket registration and RSA-PSS-signed message persistence, backend shutdown/recovery, and continued writes through the remaining backends.
- A concurrent WebSocket duplicate-delivery test using a fake notification broker.
- One-batch, ordered storage-retrieval tests for the message-bus batched query path.
- The bounded hosted HTTP-plus-distributed-WebSocket qualification described in Section 6.

The critical invariants checked throughout were: durable acknowledgement of every accepted write; zero duplicate message IDs under concurrent retry; identical feed content and hash whether read through the load balancer or directly from a backend; correct, strictly increasing sequence order; correct backend removal on failure and correct automatic recovery; and receipt of every WebSocket probe message on observers connected to all three backend processes independently.

Short final summary

Three backends behind a health- and performance-aware Go load balancer, one shared PostgreSQL database, AES-GCM encrypted persistence, and idempotent duplicate-safe writes: the required architecture is complete, correct, and verified end to end. The single unresolved constraint is delivery of the entire, ever-growing feed under high concurrency, which no connection cap, buffer size, or caching policy tried in this project removes – only a change to the feed's delivery contract plausibly would. The best complete official result was Test 18 (1.07% static error, 758.07 ms mean); the best latency was Test 23 (566.83 ms), at the cost of a diagnosed and since-fixed, but not re-submitted, breakpoint failure. After the leaderboard session, the previously process-local secure chat was extended with a batched PostgreSQL event bus so that a message committed through one backend is broadcast live to clients connected to any other, validated at bounded scale. Every number in this report traces to a committed load-generator artefact, an official evaluator capture, or `docs/optimization-log.md`.

A Figure reproduction

The plotted subset of the custom-generator's representative local run is committed at `report/local-final-run.j` the local-run figures and both architecture diagrams are recreated with:

```
python3 report/generate_figures.py
python3 report/generate_arch_figures.py
```

The official-leaderboard and WebSocket-batching figures are recreated from the committed raw evaluator captures under `leaderboard-results/` with:

```
python3 report/generate_report_figures.py
```

B Code and experiment traceability

The complete implementation history is recorded in Git. Detailed chronological experiment notes – including every numeric result quoted in Section 15, and many more experiments not summarised in this report – are maintained in `docs/optimization-log.md`; deployment and full API instructions are in `README.md`; the original assignment requirements are preserved in `docs/lab6.md`; and raw local/leaderboard JSON and CSV result files remain in their respective (git-ignored) result directories.

C Historical: the removed commit-serialisation diagnostic

Section 15 describes a PostgreSQL advisory-lock candidate that provably eliminated the feed-cache out-of-order-commit reset pathology. Unlike every other code block in this report (Section 12), this one is *not* present in the final tree – it is reproduced here purely for the record, because its measurement (Table 3's neighbouring discussion) is real and worth keeping even though the code itself was removed in the later cleanup that also removed the feed cache.

```
WITH sequence_commit_order AS MATERIALIZED (
    SELECT pg_advisory_xact_lock(4475970, 1)
),
input AS ( ... )
INSERT INTO messages ( ... )
SELECT ... FROM deduplicated
CROSS JOIN sequence_commit_order
```

```
ORDER BY position
ON CONFLICT (message_id) DO NOTHING
RETURNING sequence, message_id, created_at;
```

This transaction-scoped advisory lock forced every backend's batched insert to allocate its `BIGSERIAL` sequence range and commit strictly in that lock's acquisition order, which eliminated the out-of-order-commit condition that otherwise forces a full feed-cache rebuild – confirmed by two independent clean qualification runs (11 resets → zero, reproducibly). Its own cross-backend write-serialisation cost outweighed that benefit above roughly 2,000 concurrent users, so it was kept as an opt-in diagnostic (`DB_SERIALIZE_SEQUENCE_COMMITS=1`) for one development cycle and then removed entirely, rather than shipped disabled by default.